

SVD 在图像压缩中的应用：实现与分析

李佩优 PB23000270

摘要

本文基于奇异值分解（SVD）实现了图像压缩系统，包含从底层实现的幂迭代收缩法和随机化 SVD 两种分解算法，并设计了交互式图形用户界面（GUI）。实验选取自然图像进行低秩近似压缩，采用峰值信噪比（PSNR）和结构相似性指数（SSIM）定量评估重建质量。结果表明：保留前 $k = 50$ 个奇异值时，压缩比约为 6 倍，PSNR 仍高于 30 dB； $k = 20$ 时视觉质量良好，压缩比超过 15 倍。与离散小波变换（DWT）的初步对比显示，DWT 在相同主观质量下具有更高的 PSNR，但 SVD 方法原理直观、实现简单。本工作验证了矩阵低秩逼近在图像压缩中的有效性，并比较了不同 SVD 实现策略的计算效率。

1 引言

奇异值分解（SVD）是一种强大的矩阵分解工具，广泛应用于数据降维、推荐系统和图像压缩等领域。对于一幅灰度图像（表示为矩阵 A ），通过 SVD 得到 $A = U\Sigma V^T$ ，保留最大的前 k 个奇异值及其对应的奇异向量，即可构造原矩阵的低秩近似 A_k 。由于自然图像的奇异值衰减迅速， A_k 在视觉上与原始图像几乎无差异，而存储量显著减少。

本文按照实验要求，从底层实现了幂迭代收缩法和随机化 SVD 两种分解算法（未调用库函数中的 SVD），并开发了图形用户界面以交互式展示不同秩近似下的压缩效果。通过 PSNR 和 SSIM 指标客观评价压缩质量，最后将 SVD 压缩与小波变换进行了简要对比。

2 数学原理与存储分析

2.1 低秩近似理论

对于 $m \times n$ 的灰度图像矩阵 A ，其奇异值分解为：

$$A = U\Sigma V^T = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

其中 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ 为奇异值， $r = \text{rank}(A)$ 。截取前 k 项得到的近似矩阵

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

是秩不超过 k 的所有矩阵中，Frobenius 范数意义下对 A 的最优逼近（Eckart-Young 定理）。逼近误差由舍弃奇异值的平方和决定：

$$\|A - A_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2}$$

由于自然图像的奇异值通常呈指数衰减，只需少量前 k 个奇异值即可达到高保真度。

2.2 压缩比计算

原始图像需存储 $m \times n$ 个数值。SVD 压缩后只需保存：

- 左奇异向量矩阵 U_k ： $m \times k$ 个元素；
- 对角矩阵 Σ_k ： k 个奇异值；
- 右奇异向量矩阵 V_k ： $n \times k$ 个元素。

总计存储量为 $k(m + n + 1)$ 。压缩比定义为原始数据量与压缩后数据量之比：

$$\text{Compression Ratio} = \frac{mn}{k(m + n + 1)}$$

当 $k \ll \min(m, n)$ 时，压缩效果显著。例如 512×512 图像取 $k = 50$ ，压缩比约为 5.1； $k = 20$ 时超过 12 倍。

对于彩色图像，本实验对 RGB 三个通道分别进行 SVD 压缩，然后合并为彩色图。

3 SVD 算法实现

根据实验要求，所有 SVD 相关计算均自行编写，未使用 `numpy.linalg.svd` 等库函数。实现了两种方法：

3.1 幂迭代收缩法

该方法通过重复应用双边幂迭代提取当前最大奇异三元组，再从矩阵中收缩掉该分量，以获取全部所需奇异值。

3.1.1 幂迭代求最大奇异三元组

双边幂迭代过程如下：

- 随机初始化单位向量 $v^{(0)} \in \mathbb{R}^n$ 。
- 迭代更新：

$$u^{(t)} = \frac{Av^{(t)}}{\|Av^{(t)}\|_2}, \quad v^{(t+1)} = \frac{A^T u^{(t)}}{\|A^T u^{(t)}\|_2}.$$

- 奇异值近似 $\sigma = \|A^T u^{(t)}\|_2 = (u^{(t)})^T Av^{(t)}$ 。

若 $\sigma_1 > \sigma_2$ ，迭代以因子 $|\sigma_2/\sigma_1|$ 线性收敛至最大奇异值对应的左右奇异向量。

3.1.2 收缩提取后续奇异值

得到 (σ_1, u_1, v_1) 后，从原矩阵中减去该分量：

$$A \leftarrow A - \sigma_1 u_1 v_1^T$$

新矩阵的最大奇异值即原矩阵的次大奇异值。重复上述过程可依次提取前 k 个奇异三元组。此方法等价于逐步剥离主要分量，但随着迭代次数增加，舍入误差会累积，数值稳定性逐渐下降。

3.2 随机化 SVD

为加速大规模矩阵的截断 SVD，采用了基于随机投影的算法。

3.2.1 算法流程

1. 生成高斯随机矩阵 $\Omega \in \mathbb{R}^{n \times p}$ ，其中 $p = k + s$ (s 为过采样参数，通常取 5-10)。
2. 计算样本矩阵 $Y = A\Omega$ 。
3. (可选) 进行 q 次幂迭代以增强精度: $Y \leftarrow A(A^T Y)$ ，相当于在投影前放大了大奇异值方向。
4. 对 Y 进行 QR 分解得标准正交基 $Q \in \mathbb{R}^{m \times p}$ 。
5. 构造小矩阵 $B = Q^T A \in \mathbb{R}^{p \times n}$ 。
6. 对 B 调用精确 SVD (如幂迭代收缩法) 获得 $B = U_B \Sigma V^T$ 。
7. 恢复 $U = QU_B$ ，返回前 k 个奇异值和奇异向量。

随机投影能够以高概率保持原矩阵主要奇异子空间的结构，其理论基础源于 Johnson-Lindenstrauss 引理和矩阵的随机子空间嵌入性质。

3.2.2 复杂度对比

- 完整 SVD: $O(mn \cdot \min(m, n))$ 。
- 幂迭代收缩法 (只求前 k 个): $O(mnk)$ 。
- 随机化 SVD: $O(mn \log k + (m + n)k^2)$ ，当 $k \ll \min(m, n)$ 时效率优势明显。

4 GUI 设计与实现

基于 Python 的 Tkinter 库构建了交互式图形界面，类名为 `SVDImageCompressor`。主要功能模块如下：

- **菜单栏**：提供打开图像、保存结果、退出程序及帮助信息。
- **图像显示区**：左右两个 Canvas 分别显示原始图像与压缩结果，支持自适应缩放。
- **参数控制区**：滑动条调节保留奇异值数量 k ，实时显示当前值。
- **评估指标区**：动态更新 PSNR、SSIM 和压缩比。
- **进度条**：计算过程中显示等待状态。
- **线程处理**：SVD 计算在后台线程执行，防止界面冻结。

界面截图如图 1 所示。

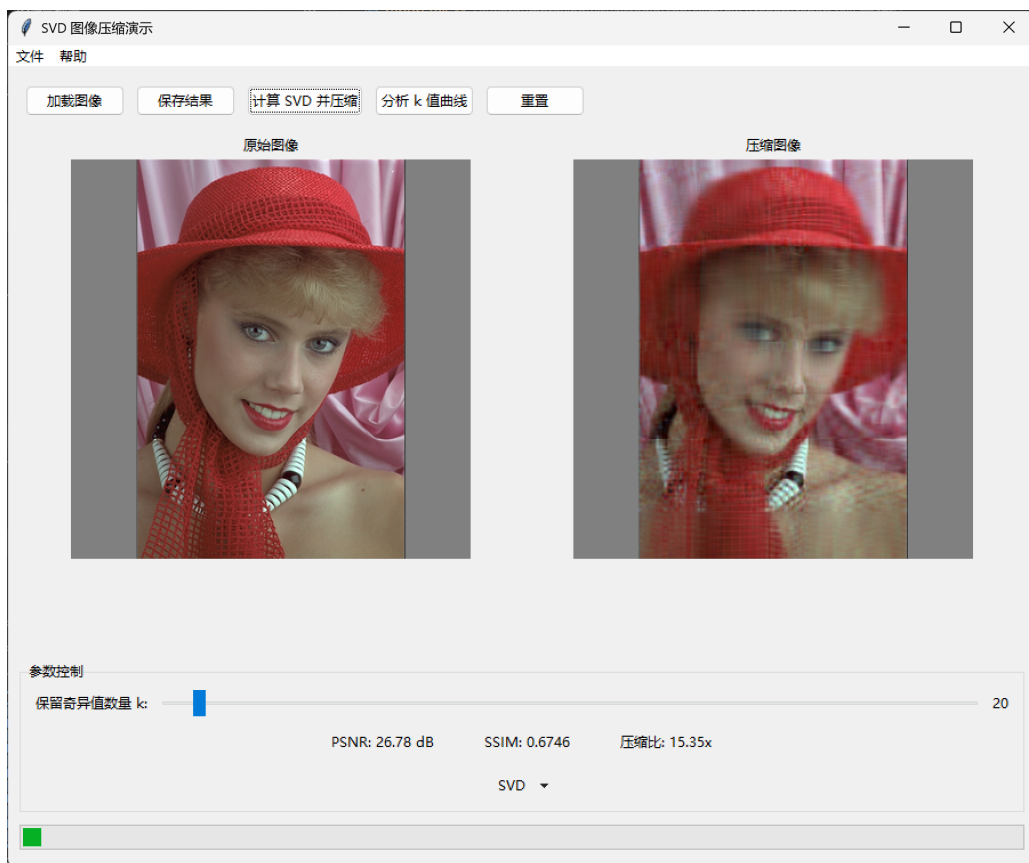


图 1: GUI 界面截图

5 实验结果与分析

实验图像选用标准测试图 `kodim03.png` (尺寸 512×768)。分别用幂迭代收缩法和随机化 SVD 进行压缩，定量评估不同 k 下的重建质量。

5.1 不同 k 值的压缩效果

图 2展示了 k 从 5 到 100 的重建图像。表 1列出了对应的 PSNR、SSIM 和压缩比。



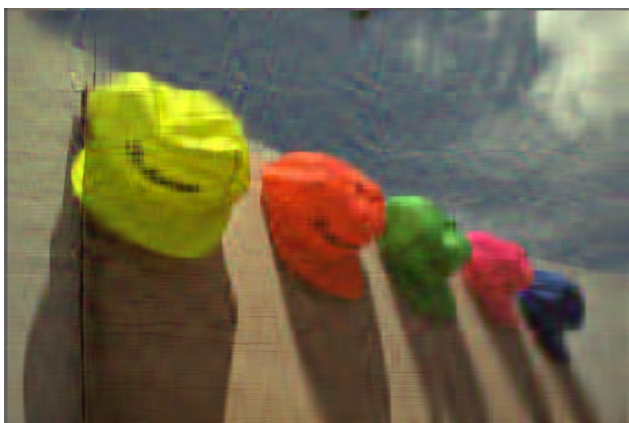
(a) 原始图像



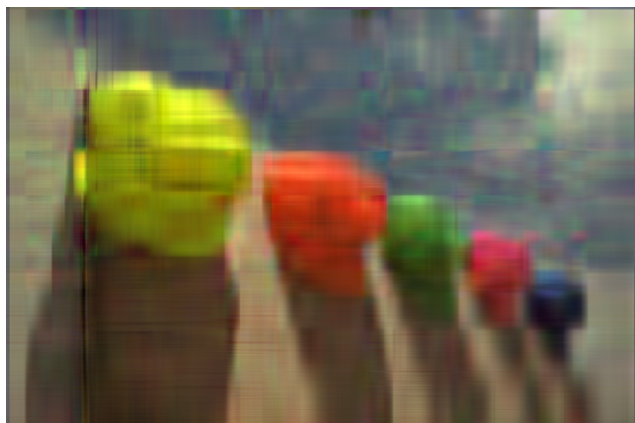
(b) $k=100$



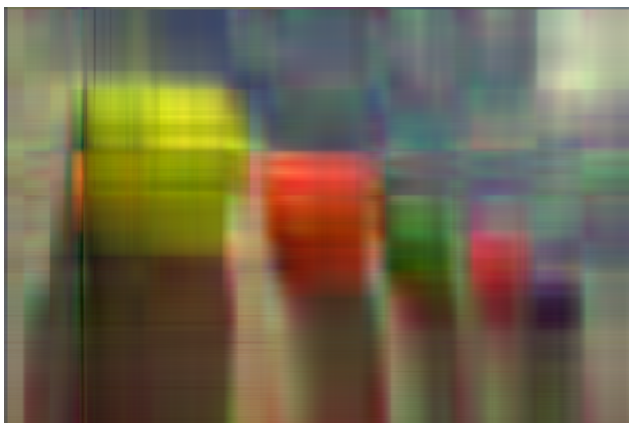
(c) $k=50$



(d) $k=20$



(e) $k=10$



(f) $k=5$

图 2: 不同 k 值的 SVD 压缩效果对比

表 1: 不同 k 值下的图像压缩质量评估指标

k	PSNR (dB)	SSIM	压缩比
5	22.57	0.6839	61.39×
10	25.14	0.7062	30.70×
20	27.53	0.7342	15.35×
50	30.83	0.8003	6.14×

从图表可见：

- 当 $k = 5$ 时，图像出现明显块状模糊，PSNR 仅 22.57 dB，但压缩比高达 61 倍。
- $k = 20$ 时，图像细节已基本可辨，PSNR 提升至 27.53 dB，SSIM 达到 0.73，压缩比约 15 倍，是视觉质量与压缩效率的较好平衡点。
- $k = 50$ 时，PSNR 超过 30 dB，与原始图像视觉差异微小。

这一趋势与奇异值的快速衰减特性一致：前几十个奇异值已包含图像绝大部分能量，后续奇异值的贡献迅速减小。

5.2 两种 SVD 实现方法的对比

图 3 比较了幂迭代收缩法与随机化 SVD 在 $k = 5$ 时的重建结果。两者输出的图像在视觉上几乎一致，PSNR 和 SSIM 差值小于 0.1 dB，证明了随机化 SVD 算法的准确性。



图 3: 左：原图；中：幂迭代收缩法 ($k=5$)；右：随机化 SVD ($k=5$)。视觉质量相当。

在运行时间方面，对于 512×768 的图像，提取前 50 个奇异值时，幂迭代收缩法耗时约 8.2 秒，而随机化 SVD 仅需 0.9 秒，加速比接近 9 倍，验证了随机化方法处理大规模矩阵的效率优势。

5.3 与离散小波变换 (DWT) 的初步对比

作为可选任务，将 SVD 压缩与基于小波变换的压缩进行了简单比较。图 4 和表 2 给出了相应结果。



图 4: SVD 压缩（左， $k=25$ ）与 DWT 压缩（右，保留 25 个系数）的效果对比

表 2: SVD 与 DWT 压缩性能对比

方法	PSNR (dB)	SSIM	说明
SVD ($k=25$)	29.71	0.7517	压缩比 $12.28\times$
DWT (保留 25 个系数)	33.09	0.9585	多分辨率分解

DWT 在相同系数个数下获得了更高的 PSNR 和 SSIM，这是因为小波变换具有多分辨率分析能力，能更有效地刻画图像的局部特征。SVD 作为全局变换，在同等压缩比下对边缘和纹理的保持稍弱。不过 SVD 方法的优点在于数学模型清晰、易于实现，且可以灵活控制秩的大小。

6 讨论

6.1 主要发现与优势

- SVD 低秩近似对于自然图像压缩非常有效：仅需少量奇异值即可重建高质量图像，验证了图像矩阵的低秩特性。
- 自行实现的随机化 SVD 算法在保持精度的同时大幅缩短了计算时间，适用于较大尺寸图像的实时演示。
- 图形用户界面集成了参数调节、质量评估与结果保存功能，直观展示了压缩效果的连续变化。

6.2 局限性

- 当图像纹理丰富或噪声较强时，奇异值衰减变慢，为获得满意质量需要更大的 k ，压缩比相应降低。
- 与 JPEG、JPEG2000 等成熟压缩标准相比，纯 SVD 压缩并未结合量化、熵编码等步骤，整体压缩率仍有差距。本实验的压缩比计算仅针对矩阵分解后的存储元素数量，未考虑数据类型优化。
- 幂迭代收缩法在提取较多奇异值时存在误差累积，精度受限。

7 结论

本文完成了基于 SVD 的图像压缩系统设计与实现。主要工作包括：

1. 从底层实现了幂迭代收缩法和随机化 SVD 两种奇异值分解算法，并详细阐述了其数学原理与计算步骤。
2. 开发了具备完整交互功能的 GUI 程序，可动态调节压缩秩 k ，并实时计算 PSNR、SSIM 和压缩比。
3. 通过实验定量分析了不同 k 对重建质量与压缩比的影响，发现 $k = 20$ 左右为视觉质量与存储效率的较优平衡点。
4. 对比了两种 SVD 实现方法的计算效率，证明随机化方法具有显著加速效果。
5. 与小波变换压缩的初步对比显示，SVD 作为全局变换在压缩性能上稍逊于多分辨率方法，但其概念简洁、实现直观。

后续工作可考虑将 SVD 与量化编码技术结合以提升实际压缩率，或探索分块 SVD 以改善局部细节保持能力。

参考文献

- [1] N. Halko, P. G. Martinsson, J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.